

This Idea stemmed from allowing non-technical users to access our data infrastructure without having to write a line of SQL. As technology is advancing and we're headed into a world where Generative AI (LLMs) are being used everywhere I wanted to incorporate an idea that I think would be great to showcase to non-technical teams in order to reduce bottlenecks around not having enough analyst to provide enough ad-hoc analysis.

Below I screenshotted 4 images of my idea to highlight that I've created a working prototype that reads the semantic layer of a dbt model, accesses the database and can generate a working sql query and output those results in real-time.

This is data model is built using the medallion architecture but incorporating the dimensional modeling as well, I believe the tool could generate even more accurate results.

Hope this idea excites you as much as it does to me!

Demo of Streamlit App

(Tool – Front-End)

(Wanted to show that you can ask it questions and the outputs)

Text-to-SQL on dbt + DuckDB + Semantic Layer

Ask a question about your data:

most recent month unique active users

Generated SQL

```
WITH recent_month AS (  
  SELECT  
    DATE_TRUNC('month', event_date) AS month_start,  
    COUNT(DISTINCT user_id) AS unique_active_users  
  FROM  
    silver_user_events  
  WHERE  
    event_date >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1 month'  
  GROUP BY  
    month_start  
)  
  
SELECT  
  month_start,  
  unique_active_users  
FROM  
  recent_month  
ORDER BY  
  month_start DESC  
LIMIT 1;
```

Query Results

	month_start	unique_active_users
0	2025-10-01 00:00:00	17751

Part of Semantic Layer

(Wanted to highlight here how it takes my measures and creates a query with that in mind)

```
! user_events_semantics.yml M x
ecommerce_data_insights > models > semantic_models > ! user_events_semantics.yml
1  version: 2
2
3  semantic_models:
4    - name: user_events
5      model: ref('silver_user_events')
6      description: "Semantic model representing cleaned user event data."
7
8      entities:
9        - name: user_id
10          type: primary
11        - name: session_id
12          type: foreign
13
14      dimensions:
15        - name: event_date
16          type: time
17          type_params:
18            time_granularity: day
19          description: "Date when the event occurred"
20
21      measures:
22        - name: active_users
23          description: "Number of unique users active during the time period"
24          agg: count_distinct
25          expr: user_id
26          agg_time_dimension: event_date
27
28        - name: purchase_events
29          description: "Count of purchase events"
30          agg: count
31          expr: event_id
32          agg_time_dimension: event_date
```

Part of Streamlit App (Ingesting database & schema.yml)

```
app.py  X
app.py > ...
1  import os
2  import re
3  import streamlit as st
4  import duckdb
5  from openai import OpenAI
6
7  # -----
8  # \ PATH CONFIGURATION (dynamic)
9  # -----
10 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
11 DB_PATH = os.path.join(BASE_DIR, "data", "ecommerce.db")
12 SEMANTIC_YAML_PATH = os.path.join(
13     BASE_DIR, "ecommerce_data_insights", "models", "semantic_models", "user_events_semantics.yml"
14 )
15
16
17 client = OpenAI(api_key=st.secrets["OPENAI_API_KEY"])
18
19
20 #
```

Part of Streamlit App (Calling to OpenAI API, SQL + Results)

```
app.py ×
app.py > ...
77
78 Rules:
79 - Return only SQL, no explanations.
80 - Prefer the most relevant tables and measures.
81 - Use proper DuckDB syntax.
82 """
83
84 # 4 Generate SQL with OpenAI
85 try:
86     completion = client.responses.create(model="gpt-4o-mini", input=query_prompt)
87     sql = completion.output_text.strip()
88
89     # Remove Markdown fences (```sql ... ```) before executing
90     sql = re.sub(r"^```[a-zA-Z]*\n|```$", "", sql.strip(), flags=re.MULTILINE)
91
92     st.subheader("🧠 Generated SQL")
93     st.code(sql, language="sql")
94 except Exception as e:
95     st.error(f"⚠️ Error generating SQL: {e}")
96     st.stop()
97
98 # 5 Run query and display results
99 try:
100     df = con.execute(sql).fetchdf()
101     st.subheader("📊 Query Results")
102     st.dataframe(df)
103 except Exception as e:
104     st.error(f"⚠️ Error executing SQL: {e}")
105
106 # 6 Clean up
107 con.close()
108
```